Politecnico di Torino

Department of Electronics and Telecommunications

Multidisciplinary Workshop

SoC Verification Strategies

Step 3: Technical Report

Filippo Brogi: s345969@studenti.polito.it
Giuseppe Maganuco: s345962@studenti.polito.it
Mateus Ferreira: s346294@studenti.polito.it

January 19, 2026

Contents

1 Introduction **2**
1.1 Reference DLX architecture . . . 2

2 Design **5**
2.1 Stages 1+2: Fetch + Decode . . . 5
2.1.1 IF + ID DUTs . . . 5
2.1.2 IFID Testbench . . . 6
2.2 Stage 3: Execute . . . 7
2.2.1 EX DUT . . . 7
2.2.2 EX Testbench . . . 7
2.2.3 Post-Synthesis . . . 8
2.3 Stages 4+5: Memory + Write Back . . . 8
2.3.1 MEM + WB DUTs . . . 8
2.3.2 MEMWB Testbench . . . 9
2.4 Control Unit . . . 9
2.4.1 CU DUT . . . 9
2.4.2 CU Testbench . . . 9

3 Results **11**
3.1 IFID . . . 11
3.2 EX . . . 11
3.2.1 RTL . . . 11
3.2.2 Post Synthesis . . . 11
3.2.3 Summary . . . 12
3.3 MEWB . . . 12
3.3.1 RTL . . . 12
3.3.2 Post Synthesis . . . 13
3.3.3 Summary . . . 14
3.4 CU . . . 14
3.4.1 RTL . . . 14
3.4.2 Post Synthesis . . . 15
3.4.3 Summary . . . 16

4 Testing and Fault Tolerance - Assignment **18**
4.1 Testbench structure . . . 18
4.2 Fault Coverage computation . . . 18
4.3 Implementation . . . 18
4.4 First Approach: Bash loop . . . 18
4.5 Second Approach: InjectFault Test class . . . 19
4.6 Pros and Cons . . . 19
4.7 Results . . . 19

5 Conclusion **21**

1 Introduction

In the third step of the workshop we started working on the Datapath and Control Units of the MIPS-like DLX CPU that was developed during the *Microelectronic Systems* course. Specifically, the task was about verifying Datapath subcomponents independently from one another, and the Control Unit. Regarding the former, we chose the following proposed scheme:

- **Fetch** + **Decode Stage** (**IFID**): This macro-block is composed of stages 1 and 2 of the original DP and is in charge of incrementing the input PC and also accessing registers inside the RF by decoding the input IR fields. There is no Instruction Memory in this stage, as it is a standalone component, placed outisde both DP and CU.
- **Execute Stage** (**EX**): This stage remained the same as in the original DP, meaning it is basically composed of an ALU (previously verified), a zero comparator for evaluating conditional branch instructions (`BEQZ`, `BNEZ`), and some MUXes for choosing which operands for the ALU operation to be performed.
- **Memory** + **Write Back** (**MEMWB**): This macro-block generates the next PC value to be registered and then passed to the IRAM to access the next progrma instruction, and also `Address` and `Data` signals to access DRAM, which is, like IRAM, a standalone component.

1.1 Reference DLX architecture

Figure 1 shows the whole DLX architecture, just to outline the overall structure and I/DRAM components being outside the DP:

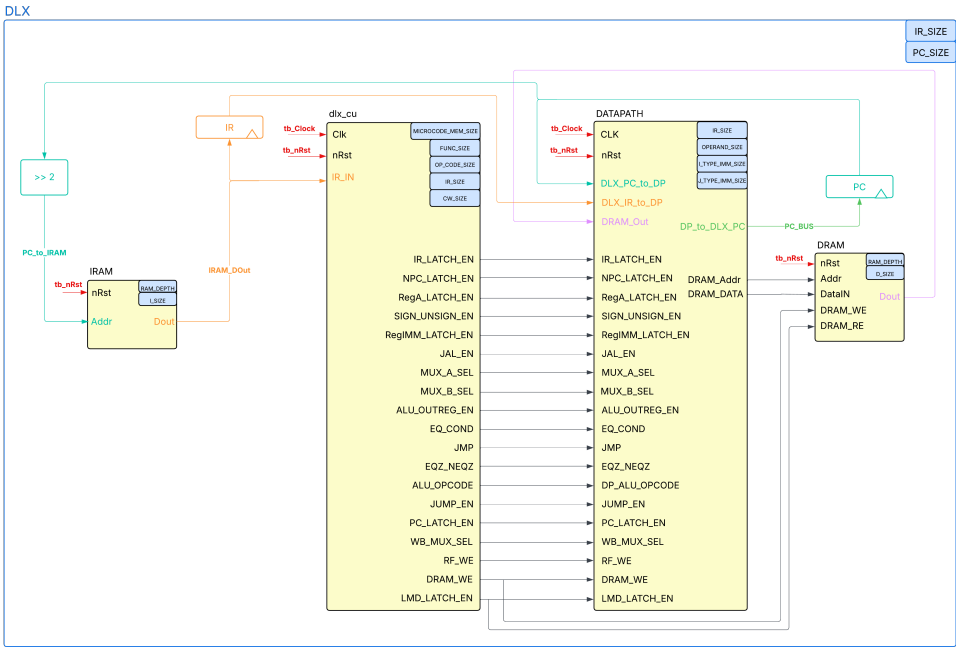


Figure 1: DLX Top Level

Figure 2 shows the DLX CU:

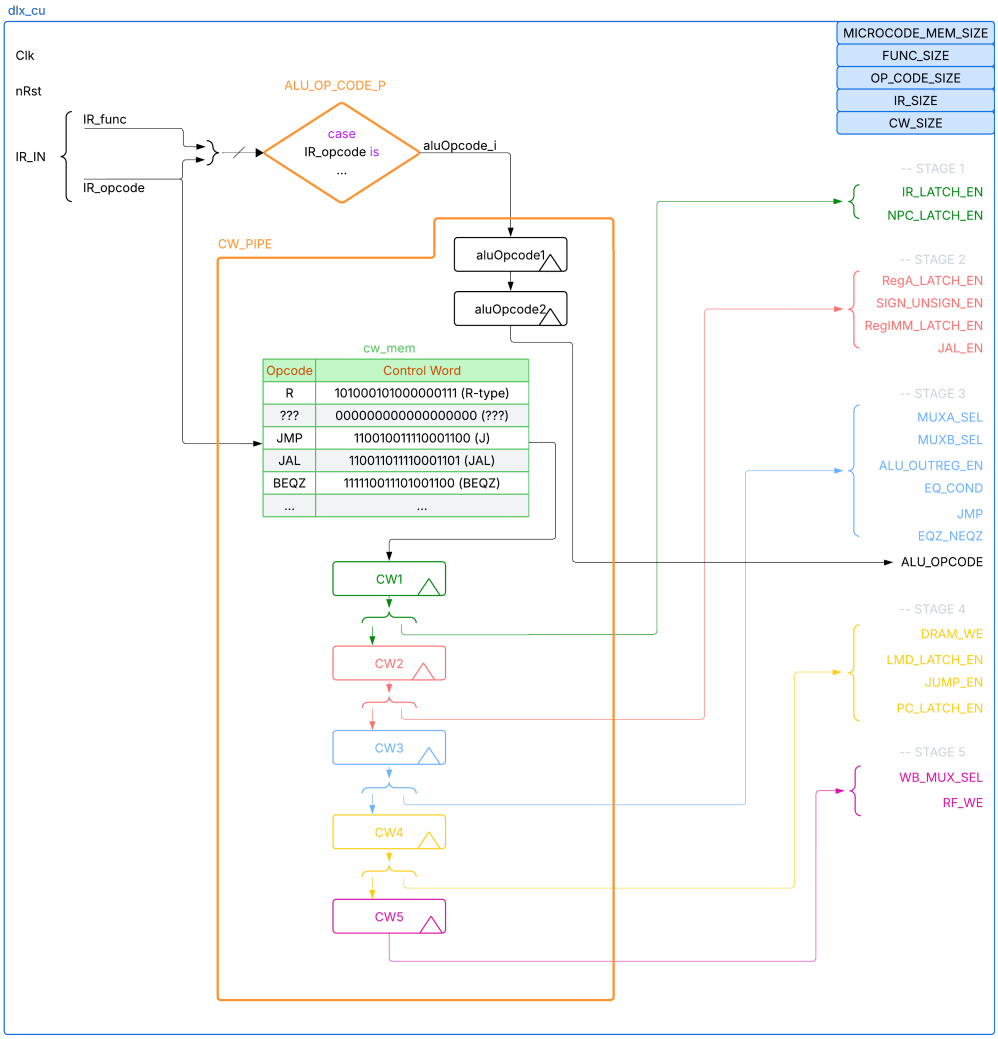


Figure 2: DLX Control Unit

Figure 3 shows the whole original DLX DP:

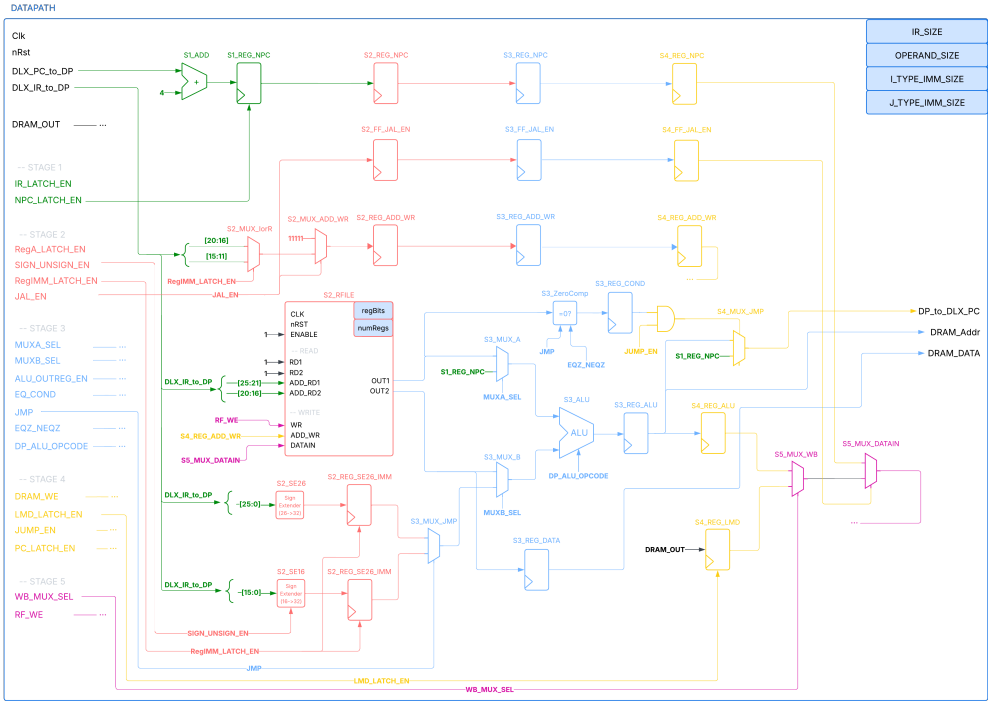


Figure 3: DLX Datapath

2 Design

In this section each DP sub-block (DUTs) will be analyzed and, for each one, the main highlights from the corresponding testbench will be described. Aside from the CU, since the original DLX DP was built structurally, although with clear distinction of stages, the objective of this section is to simply explain what each block in Figure 2 and Figure 3 do.

As a general note, every UVM testbench in this step is:

- **Multi Sequence**: One "standard" Sequence Item class which implements item fields, and multiple Sequence classes, each specialized in a specific RCG.
- Single Agent
- Single Environment
- **Single Test**: Since there are multiple Sequences, the Test class creates and instantiates every defined Sequence and starts them in parallel using the `fork ... join` construct, on the same Sequencer.

Only the notable differences with respect to the previously developed testbenches will be highlighted. For more details, refer to the source code.

2.1 Stages 1+2: Fetch + Decode

2.1.1 IF + ID DUTs

With reference to Figure 3, the Fetch + Decode Stage:

- Increments the PC to address the next instruction inside IRAM.
- Decomposes the IR input into its fields.
- Drives `S2_MUX_ADD_WR` which lets through either $(31)_{10}$ (Link Register RF address) or another register address specified inside the instruction fields, to later use it as Write Address to access the RF.
- Sign-extends (depending on the instruction type) an immediate value placed in the instruction.

When encapsulating both stages into a single VHDL entity, some additional input signals from later stages are added, and they are reported here:

```
    -- Additional RF inputs (STAGES 4 and 5)
    RF_WE              : in    std_logic;
    S4_REG_ADD_WR_OUT  : in    std_logic_vector(log2(RF_numRegs) - 1
        downto 0);
    S5_MUX_DATAIN_OUT  : in    std_logic_vector(RF_regBits - 1 downto 0);
```

Listing 1: Additional input signals

2.1.2 IFID Testbench

Package Inside the **pkg_constants.sv** package there are instruction aliases, which are `structs` defining each instruction type's fields. Everything is enclosed within a `union` (called `InstrType`) which allows to access every possible instruction type field, depending on the need.

I, J, and R-Type OPCODEs, along with R-Type FUNC fields are defined via constants inside the package.

Then, three utility functions are defined:

- `check_instr_type()`: Given an instruction, returns its type (I, J, R).
- `check_instr_sign()`: Given an instruction, returns whether it interprets the immediate field as signed or unsigned.
- `get_instr_type()`: Given an instruction, returns an instruction alias of that type.

Sequence Each Sequence Item also has an `InstrType` alias field which was meant to be used in CRG to access instruction fields in a cleaner way rather than hardcoding IR bit slices. Unfortunately, it is not possible to do so

Six Sequences are defined:

- **I-Type Sequence**: A Sequence in which only I-Type Sequence Item instructions are generated by the randomizer.
- **J-Type Sequence**: A Sequence in which only I-Type Sequence Item instructions are generated by the randomizer.
- **R-Type Sequence**: A Sequence in which only I-Type Sequence Item instructions are generated by the randomizer.
- **Random Sequence**: Here every Sequence Item field is constrained randomically, and can thus yield invalid inputs to the IFID DUT.
- **Unsigned Addition Sequence**: Sequence in which only unsigned addition instructions are generated, focusing on verifying overflow cases.
- **Signed Addition Sequence**: Sequence in which only unsigned addition instructions are generated, focusing on verifying underflow cases.

Golden Model The Golden Model was implemented inside the Scoreboard by mimicking the DUT behavior through arrays, queues and SV `static` variables, allowing them to persist across mulitple clock cycles. Here are some notable examples:

```
    // Mimick "NPC = NPC + 4" combinational behavior
    Expected_S1_ADD_OUT     = ifid_seqitem.DLX_PC_to_DP + 'd4;
    [...]
    /* Mimick registers S1/2_REG_NPC_OUT through a queue */
    Expected_S1_REG_NPC_OUT = s1_reg_queue[0];  // Get last CC's NPC
        value
    Expected_S2_REG_NPC_OUT = s1_reg_queue[1];  // Get NPC of 2 CCs ago
    s1_reg_queue.push_front(Expected_S1_ADD_OUT);  // Push new NPC into
        first position
    [...]
    /* Mimick RF (synchronous R/W) behavior through an array */
```

```
    // Write operation
    if (ifid_seqitem.RF_WE) begin
      rf[ifid_seqitem.S4_REG_ADD_WR_OUT] = ifid_seqitem.S5_MUX_DATAIN_OUT
          ;
    end
    // Read operations (always enabled since RD1=1 and RD2=1)
    Expected_S2_RFILE_A_OUT = rf[ifid_seqitem.DLX_IR_to_DP[25:21]];
    Expected_S2_RFILE_B_OUT = rf[ifid_seqitem.DLX_IR_to_DP[20:16]];
```

Listing 2: DUT Mimicking logic

2.2 Stage 3: Execute

2.2.1 EX DUT

With reference to Figure 3, the Exe in the Execution unit is where arithmetical and logical operations get executed by the ALU, and also where branch conditions get evaluated. This unit handles the following opeations:

- Computing branch conditions
- Selecting the correct Program Counter to pass to the next stage (either the incremented one or the target jump address)
- Arithmetical operations such as (un)signed additions and subtractions
- Logical operations such as arithmetical or logical shifts/rotates, both left and right
- Computing the address for the data memory

2.2.2 EX Testbench

The Sequencer is the one who defines how the random stimuli are generated, so is the real entity that determines the coverage. Inputs are generated randomly. The DUT is mainly stimulated by the ALU opcode, and by driving it accordingly, it is possible to stimulate the majority of the EX Stage. Given the high number of ALU Opcodes, different kinds of constraints were applied, to guarantee high coverage. As usual, each Sequence specializes in implementing just one kind of constraint.

The main combination are:

- Totally random constraints
- Arithmetic unit constraints
- Logic unit constraints
- Shifter unit constraints
- Comparator unit constraints for stimulating the following conditions:
  - Equal true
  - Not equal true
  - Greater than

  - Lesser than

- Branch unit constraints for stimulating the following conditions:
  - Jump equal
  - Jump not equal
  - Greater than
  - Lesser than

2.2.3 Post-Synthesis

Delay During post-synthesis netlist simulation, the DUT requires an additional clock cycle. In the first iteration, the output is always undefined (X) for any input; therefore, an extra delay is introduced to allow the design to stabilize.

Golden Model The Golden Model was implemented inside the Scoreboard by mimicking the DUT behavior, following the same approach as for the IFID Testbench. What follows is some notable mimicking logic code snippets:

```
    // -------- Mimick register behavior --------
    if (exe_seqitem.nRST == 1) begin
            Expected_S3_REG_ALU_OUT         = syn_prev_S3_REG_ALU_OUT;
            Expected_S3_REG_DATA_OUT        = syn_prev_S3_REG_DATA_OUT;
    [...]
    // ----------- Mimick ALU behavior ----------
    if (exe_seqitem.DP_ALU_OPCODE == SLL_op)
            S3_ALU_OUT = Shifter_Result;
    else if (exe_seqitem.DP_ALU_OPCODE == SRL_op)
    [...]

    prev_S3_REG_ALU_OUT    = (exe_seqitem.nRST == 1) ? S3_ALU_OUT : 0;
    prev_S3_REG_DATA_OUT   = (exe_seqitem.nRST == 1) ? exe_seqitem.
        S2_RFILE_B_OUT : 0;
    prev_S3_FF_COND_OUT    = (exe_seqitem.nRST == 1) ? BranchTaken : 0;
    prev_S3_FF_JAL_EN_OUT  = (exe_seqitem.nRST == 1) ? exe_seqitem.
        S2_FF_JAL_EN_OUT : 0;
    prev_S3_REG_NPC_OUT    = (exe_seqitem.nRST == 1) ? exe_seqitem.
        S2_REG_NPC_OUT : 0;
    prev_S3_REG_ADD_WR_OUT = (exe_seqitem.nRST == 1) ? exe_seqitem.
        S2_REG_ADD_WR_OUT : 0;
```

Listing 3: DUT Mimicking logic

2.3 Stages 4+5: Memory + Write Back

2.3.1 MEM + WB DUTs

With reference to Figure 3, the MEM + WB Stage, this Stage is the one that will access to read and write the Data Memory and, if needed, write inside the RF the new value.

2.3.2 MEMWB Testbench

Sequence The MEMWB Block is completely combinational besides pipelining registers. There are no hard constraints on the generated input stimuli, and regarding multiplexers, all possible selection cases are equally important to verify, therefore a Sequence consists only of random inputs without any probabilistic constraints.

Golden Model The golden model is structured as follows:

- The **predictor** function takes as input the transaction packets, which contain the stimulus and response of the transaction (in this case the signals), and creates a predictor packet which predicts the response of the transaction packet.
- The testbench only compares the output of the transaction packet and the predictor packet. If they differ, the testebench throws an error.

A note about post-synthesis simulation The testbench runs into timing issues when an SDF file is involved.

2.4 Control Unit

2.4.1 CU DUT

DUT Architecture With reference to Figure 2, the CU DUT is composed of a Control Word Memory (`cw_mem`) which gets indexed with the current instruction's `OPCODE` field, producing the corresponding CW for the DP. As the architecture is pipelined, Pipeline registers $\texttt{CW}_i$ are introduced to allow both each stage to receive the CW at the right time, and also to allow the possibility of having multiple instructions flowing in the DP at the same time in different stages.

DUT Source code adjustments The DUT source code was slightly modified in two ways. First, since the CU DUT was using an enumerated type (`aluOp`) which serves to encode ALU Opcodes as an output signal, which is incompatible with mixed SystemVerilog+VHDL modules connections. The solution was to simply encode explicitly, rather than implicitly as was done previously, the `ALU_OPCODE` CU output as `std_logic_vector(NUM_ALU_OPCODES-1 downto 0)` and convert, from the enumerated VHDL type `AluOp` used for all CW signals inside the architecture, to the new bit array type that the module was now exposing.

The second modification was about timing, as when simulating the DUT, it always produced Xs as CW outputs. This was due to there being 2 processes, one sequential one which shifted CWs between pipeline registers, and another one, combinational, which was sampling (and changing!) the input ALU Opcode (and so the output) of the CW Memory. This architecture, although working fine when simulating the CU DUT in the original DLX CPU, exposed some problems when placing the block inside SV simulations. The adopted solution was to simply make both processes sequential, so that the CW Memory's output cannot change while the "shift process" is trying to shift (previously unstable) register values through the register chain.

2.4.2 CU Testbench

Package The package was improved from the IFID version by including a 1:1 match of the DUT CW Memory, and two further functions:

- `get_cw()`: Given an instruction, returns the correspoding CW by indexing the CW Memory.
- `get_aluop()`: Given an instruction, returns the ALU Opcode that the ALU needs to perform for executing that instruction.

Sequence The structure is the same as in the IFID Testbench: one Sequence Item, multiple Sequences.

Four Sequences are defined, with one being the (fully) **Random Sequence**, and the others being **I-Type Sequence**, **J-Type Sequence** and **R-Type Sequence**, which generate only the corresponding instruction types, with legal instruction fields.

Golden Model The Golden Model was implemented very simply by using the two additional functions defined in the package, as it basically needed to generate the entire CW and match it against the DUT-generated one, and the same for the ALU Opcode.

3 Results

3.1 IFID

Coverage results for the IFID Testbench are reported in the following table ($n = 1000$ Sequence Items):

	Branch	Condition	Expression	Statement	FSM	Toggle	Functional
RTL	96.00%	100.00%	N.D.	96.90%	N.D.	91.19%	100.00%
Post-Synthesis	87.50%	N.D.	N.D.	94.89%	N.D.	93.78%	100.00%

Table 1: IFID Block RTL vs Post-synthesis coverages

3.2 EX

In this subsection we'll describe the result obtained, and differentiate it in two different sections: Register-Transfer Level (RTL) description coverage and Post Synthetis coverage. For both of them we tried to reach the highest possible coverage

3.2.1 RTL

For the RTL descriptin, the highest coverage reached is 95.65%. Isn't possible to reach an higher coverage due to two characteristics of the how the RTL was written:

- Hardcoded register enable, set to 1
- Unitilized part of shifter

In particular, when talking about the shifter, it integrates two function, the shift function (both arithmetic and logic, both left and right) and the rotate. But the rotate function isn't used by any instruction and so from the EX unit of the Datapath isn't possible to reach the rotate part in the shifter.

To understand how much the unuded part of shifter affect the coverage, a test with an optimized shifter is done, and in this way, the coverage became even higher, with 98.15%, with a $\Delta_{cov} = 1.5\%$

3.2.2 Post Synthesis

Using the same stimuli, we're able to reach a coverage of 95.09%, and the optimized version (without rotate), we're able to reach 95.9%, this time only a $\Delta_{cov} = 0.81\%$.

To reach a greater coverage, we've analyzed the unreached toogle, and the most of them were $S3_ALU_sub_73_*$. So the test was modified to stimulate the sub logic circuit even more, but the coverage didn't changed. So we've give a look to the source code, and analyzed the un reached togle, but the discovered piece of code, didn't helped to reach an even higher coverage, since apparently it is already stimulated, since the $S3_ALU_sub_73_*$ are stimulated by $S3_MUX_B_OUT$, which is used with a 100% probability. So all of the stimuli that aim to increase the coverage for these toogle are useless and we were not able to increase the coverage.

Figure 4: Coverage of RTL description of EXE

Coverage Summary By Instance (96.65%)

Instance	Branches	Conditions	Statements	Toggles	Total
Total	92%	100%	94.91%	99.68%	96.65%
DP_EXE_inst	92%	100%	94.91%	99.68%	96.65%

Recursive Hierarchical Coverage Details (96.65%)

Coverage Type	Bins	Hits	Misses	Coverage
Branches	100	92	8	92%
Conditions	4	4	0	100%
Statements	59	56	3	94.91%
Toggles	966	963	3	99.68%

Figure 5: Coverage of the shifter of EXE

shifter	66.66%	57.14%	100%	74.6%

3.2.3 Summary

At the end the reached coverage is:

- RTL: 95.65%
- RTL Optimized: 98.15%
- Post Synthesis: 95.09%
- Post Synthesis Optimized: 95.90%

3.3 MEWB

3.3.1 RTL

For the RTL simulation, the reduced coverages are caused by the hardcoded enable signal on all but one register. There is an additional thing bringing down the toggle coverage, for some reason questasim fails to consider the falling edge of the reset signal at the beginning of the simulation

Figure 6: Coverage of RTL description optimized of EXE

Coverage Summary By Instance (98.15%)

Instance	Branches	Conditions	Statements	Toggles	Total
Total	94.68%	100%	98.24%	99.68%	98.15%
DP_EXE_inst	94.68%	100%	98.24%	99.68%	98.15%

Recursive Hierarchical Coverage Details (98.15%)

Coverage Type	Bins	Hits	Misses	Coverage
Branches	94	89	5	94.68%
Conditions	4	4	0	100%
Statements	57	56	1	98.24%
Toggles	966	963	3	99.68%

Figure 7: Coverage of Post Synthesis of EXE

Local Instance Coverage Details (95.09%)

Coverage Type	Bins	Hits	Misses	Coverage
Toggles	6094	5795	299	95.09%

Figure 8: Coverage of Post Synthetis optimized of EXE

Recursive Hierarchical Coverage Details (95.9%)

Coverage Type	Bins	Hits	Misses	Coverage
Toggles	5098	4889	209	95.9%

3.3.2 Post Synthesis

For the postsynthesis design, the same discourse of the reset signal applies. However, the reduced coverage is caused by the fact that, for every 2-by-1 multiplexer, there seems to be two additional unused signals, which indicates that the technology library's smallest multiplexer is probably a 4-by-1 multiplexer, and two of the signals are hardwired to 0. As a matter of fact, we report here the Gate-Level netlist portion which generates signal `S3_ALU_sub_73_n15`:

Figure 9: Unreached toogles of EXE

SCALAR Signal / Value	Hits ->1	Hits ->0	Coverage
S3_ALU_sub_73_n15	0	0	0%
S3_ALU_sub_73_n16	0	0	0%
S3_ALU_sub_73_n17	0	0	0%
S3_ALU_sub_73_n18	0	0	0%
S3_ALU_sub_73_n19	0	0	0%
S3_ALU_sub_73_n20	0	0	0%
S3_ALU_sub_73_n21	0	0	0%
S3_ALU_sub_73_n22	0	0	0%
S3_ALU_sub_73_n23	0	0	0%
S3_ALU_sub_73_n24	0	0	0%
S3_ALU_sub_73_n25	0	0	0%
S3_ALU_sub_73_n26	0	0	0%
S3_ALU_sub_73_n27	0	0	0%
S3_ALU_sub_73_n28	0	0	0%
S3_ALU_sub_73_n29	0	0	0%
S3_ALU_sub_73_n30	0	0	0%
S3_ALU_sub_73_n31	0	0	0%
S3_ALU_sub_73_n32	0	0	0%
S3_ALU_sub_73_n33	0	0	0%
S3_ALU_sub_116_carry[1-31]	1	1	100%

```
INV_X1 S3_ALU_sub_73_U10 ( .A(S3_MUX_B_OUT[23]), .ZN(
    S3_ALU_sub_73_n15) );
```

Listing 4: Unreached EXE toggles culprit

3.3.3 Summary

At the end the reached coverage ($n = 1000$ Sequence Items) is:

	Branch	Condition	Expression	Statement	FSM	Toggle	Functional
RTL	80.95%	N.D.	100.00%	100.00%	N.D.	79.06%	100%
Post-Synthesis	N.D.	N.D.	N.D:	N.D.	N.D.	87.89%	N.D.

Table 2: MEMWB Block RTL vs Post-synthesis coverages

3.4 CU

3.4.1 RTL

After trying to stimulate the CU with a different combination of different stimuli aimed to generate all of the type of instructions to excite as much as possible the CU, the maximum reachable percentage was 97.94%, mostly limited by toogle coverage.

After analyzing the coverage output, the part that wasn't covered is aluOpcode3, so after analyzing the CU source code, we discovered that this signal was declared and never used.

After the optimization of the source code, in fact, the reached coverage was 100%

Figure 10: Coverage of RTL description of CU

Coverage Summary By Instance (97.94%)

Instance	Branches	Statements	Toggles	Total
Total	100%	100%	93.82%	97.94%
DP_CU_inst	100%	100%	93.82%	97.94%

Recursive Hierarchical Coverage Details (97.94%)

Coverage Type	Bins	Hits	Misses	Coverage
Branches	52	52	0	100%
Statements	84	84	0	100%
Toggles	340	319	21	93.82%

Figure 11: Unreached coverage of CU

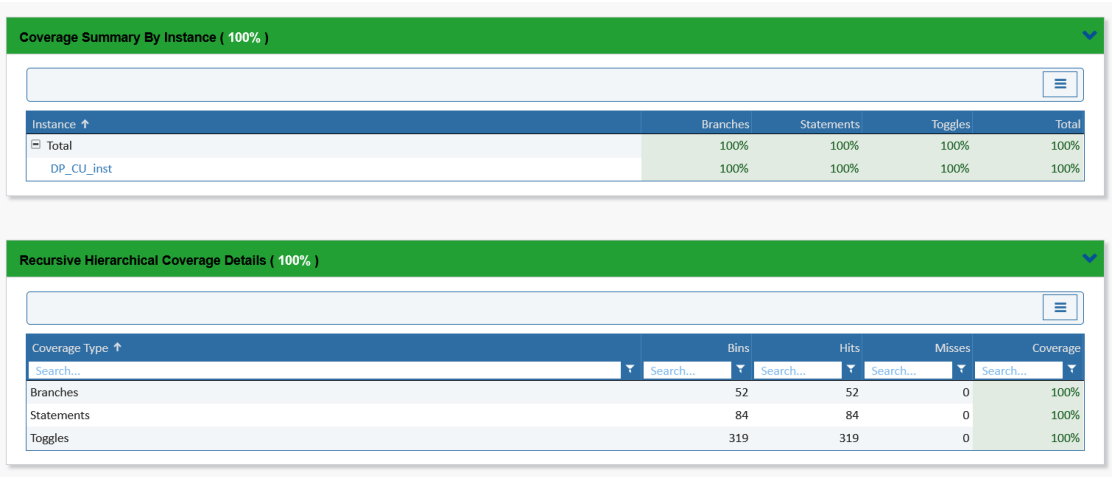
ENUM Signal / Value	Hits	Coverage
aluOpcode1		
aluOpcode2		
aluOpcode3		
NOP	0	Not Covered
ADD_op	0	Not Covered
AND_op	0	Not Covered
OR_op	0	Not Covered
SGE_op	0	Not Covered
SLE_op	0	Not Covered
SLL_op	0	Not Covered
SNE_op	0	Not Covered
SRL_op	0	Not Covered
SUB_op	0	Not Covered
XOR_op	0	Not Covered
SRA_op	0	Not Covered
ADDU_op	0	Not Covered
SUBU_op	0	Not Covered
SEQ_op	0	Not Covered
SLT_op	0	Not Covered
SGT_op	0	Not Covered
SLTU_op	0	Not Covered
SGTU_op	0	Not Covered
SLEU_op	0	Not Covered
SGEU_op	0	Not Covered
aluOpcode_i		

3.4.2 Post Synthesis

After the synthesis of the CU, the same test was applied to it, this time there wasn't any difference between the optmized and non optimized, this should be related to the fact that the synthethizer would optimize by himself the code, so the unused signal isn't a problem.

This time even with the optimization made by the synthetizer the maximum coverage reached is only 88.41%. But when checking the non covered points, it was mainly the main outputs. This

Figure 12: Optimized coverage for RTL of CU



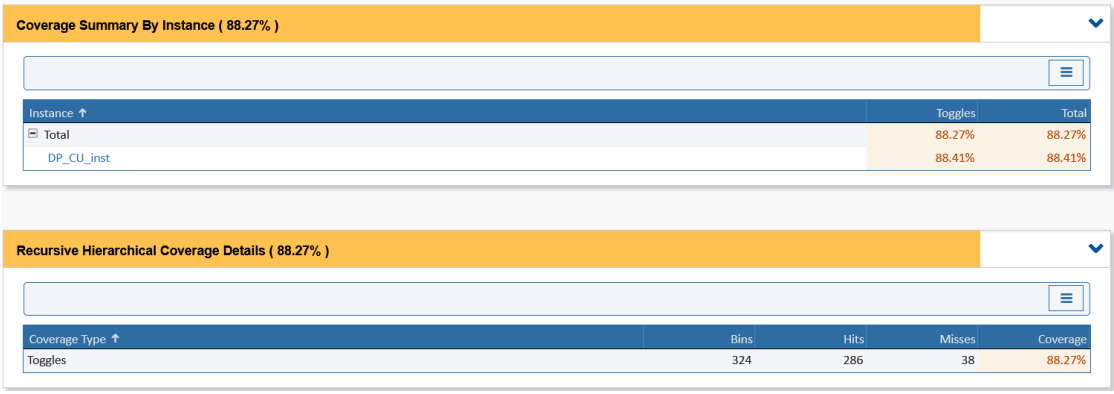
Coverage Summary By Instance (100%)

Instance	Branches	Statements	Toggles	Total
Total	100%	100%	100%	100%
DP_CU_inst	100%	100%	100%	100%

Recursive Hierarchical Coverage Details (100%)

Coverage Type	Bins	Hits	Misses	Coverage
Branches	52	52	0	100%
Statements	84	84	0	100%
Toggles	319	319	0	100%

Figure 13: Post Synthesis coverage of CU



Coverage Summary By Instance (88.27%)

Instance	Toggles	Total
Total	88.27%	88.27%
DP_CU_inst	88.41%	88.41%

Recursive Hierarchical Coverage Details (88.27%)

Coverage Type	Bins	Hits	Misses	Coverage
Toggles	324	286	38	88.27%

result was unexpected and we weren't able able to increase it, even due to the fact that the RTL coverage reached the 100%

3.4.3 Summary

Coverage results for the CU Testbench are reported in the following table ($n = 1000$ Sequence Items):

	Branch	Condition	Expression	Statement	FSM	Toggle	Total
RTL	100%	N.D.	N.D.	100%	N.D.	93.28%	97.94%
RTL optimized	100%	N.D.	N.D.	100%	N.D.	100%	100%
Post-Synthesis	N.D.	N.D.	N.D.	N.D.	N.D.	88.41%	88.41%

Table 3: CU Block RTL vs Post-synthesis coverages

Figure 14: Unreached coverage of CU Post Synthesis

Toggles Coverage (88.41%)

Page Size: 25

SCALAR Signal / Value	Hits ->1	Hits ->0	Coverage
ALU_OPCODE[0-4]	1	1	100%
ALU_OUTREG_EN	0	0	0%
Clk	1	1	100%
DRAM_WE	0	0	0%
EQZ_NEQZ	0	0	0%
EQ_COND	0	0	0%
IR_IN[0-31]	1	1	100%
IR_LATCH_EN	0	0	0%
JAL_EN	0	0	0%
JMP	0	0	0%
JUMP_EN	0	0	0%
LMD_LATCH_EN	0	0	0%
MUXA_SEL	0	0	0%
MUXB_SEL	0	0	0%
NPC_LATCH_EN	0	0	0%
PC_LATCH_EN	0	0	0%
RF_WE	0	0	0%
RegA_LATCH_EN	0	0	0%
RegIMM_LATCH_EN	0	0	0%
SIGN_UNSIGN_EN	0	0	0%
WB_MUX_SEL	0	0	0%
aluOpcode1[0-4]	1	1	100%
aluOpcode_i[0-4]	1	1	100%
n2	1	1	100%
n5	1	1	100%

1 to 25 of 121 — First Previous Page 1 of 5 Next Last

4 Testing and Fault Tolerance - Assignment

4.1 Testbench structure

The goal of the assignment is to modify the already developed testbench, enhancing it, allowing to perform a fault simulation. We chose, among the many existing fault models, the Stuck-at Fault (SAF) Model.
The testbench was modified to allow RTL, post-synthesis fault-free simulation, and post-synthesis (Stuck-At) fault simulation. After every simulation, functional and code coverage are reported. If fault simulating, also Fault Coverage is reported and an output file is created, containing the campaign's injected faults, labeled as "DETECTED" or "UNDETECTED". Since the testbenches created in precedence aimed to test the single blocks, we've performed the fault simulation with the same consideration, so we isolated: IFID, EX, MEMWB and CU, and tested them individually. All of the fault simulations are performed with single Stuck-at fault, so at the end of each simulation, it is released and a new one is injected.

4.2 Fault Coverage computation

Fault Coverage was computed at the end of every fault injection campaign by using the following formula, implemented inside a Bash script:

$$FaultCoverage\ (\%) = \frac{\#Detected\ Faults}{\#Total\ Injected\ Faults} \cdot 100$$

As seen in theoretical classes, a more accurate metric could be obtained by placing at the denominator $\#Detectable\ Faults$, which is obtained by handpicking Detectable Faults from the TetraMAX fault list in IFID and CU units. As will be explained later, EX and MEMWB units use a different custom approach to generating faults, and thus could not do so too.

4.3 Implementation

To inject a Stuck-At fault during simulation, SystemVerilog (SV) provides the couple of commands **force** and **release**, but with an important limitation, which is that arguments (signal hierachy path and value to inject onto) must be static at compile-time. For this very reason, this first implementation was scraped.
Universal Verification Methodology (UVM) provides its own way to inject fault: **uvm_hdl_force** and **uvm_hdl_release**, this time the research of the pathname is done at runtime, hence it accepts the use of variables as parameters.

When developing the Tetsbench structure we developed two solutions, the second one faster than the first one, which will now be briefly explained.

4.4 First Approach: Bash loop

Inside the "'run.sh"' Bash script, RTL simulation, post-synthesis fault-free simulation and post-synthesis fault-simulation are handled according to the script's second parameter ("'postsyn"' or "'faultsim"', blank for RTL). If fault simulation is enabled, the script exports some environment variables, which function as "inputs" to the testbench, and also defines some additional compile-time flags which get then processed by the some newly-added testbench logic to conditionally

execute a fault-free simulation or a fault-simulation. The testbench also reads from a Bash-specified file the faults which need to be injected, and after the fault injection (and simulation) campaign produces a "Classified Faults" file (“‘classified_faults.txt“‘) in which injected faults are classified as either "DETECTED" or "UNDETECTED". Finally, Fault Coverage is computed and printed to the screen.

4.5 Second Approach: InjectFault Test class

To allow the fault simulation another test was created. This will run the same sequence of stimulus for each fault. These stimulus are the same used to perform the simulation coverage.
The checks are performed by a fault-aware scoreboard, which will notify the test class if a fault is detected. The test class then interrupts the sequence if a fault was detected, and writes to a file whether it was detected or not. The test class is responsible for reading from a file and applying the stuck-at fault.
The steps of the fault simulation are:

1. The fault list is generated in TetraMax (or alternatively generated by our script)
2. The simulation start the new test
3. for each fault:
   - The fault is injected
   - The stimulus are generated
   - The scoreboard checks if there are mismatch and notify the mismatch
   - Fault is released
4. Fault coverage is printed and in a file there is the list of fault detected/not detected.

This implementation is faster due to the fact that everything happens inside of a single simulation, with only one component being iterated instead of the entire simulation. The overhead due to re-launch the simulation is avoided.

4.6 Pros and Cons

Performing fault simulation using this approach presents a first big problem. The problem is that there isn't a native solution to generate the possible faults. In fact the fault generation is performed by TetraMax, and then the fault are converted in a way that the testbench is able to inject it: $Module_topTestbench.exe_toplevel.DP_EXE_inst.S3_BranchTaken1'b0$
The main advantage is the reuse of stimulus, since to perform the functional coverage, a sequence of stimuli to stimulate as much as possible the circuit is done, the same sequence is reused for the fault simulation. But this approach even if doesn't allow diagnosis, it isn't capable to understand what fault caused the failure.

Other than that, when trying to use TetraMax to perform fault simulation on the blocks, the presence of registers at the end, will require a sequential simulation, that without a scan-chain will make the simulation-time too big. Instead performing a fault simulation within SV/UVM simulation, the presence of a golden model that create expected values, reduce the time required.

4.7 Results

Fault injection campaign results are shown in image 15

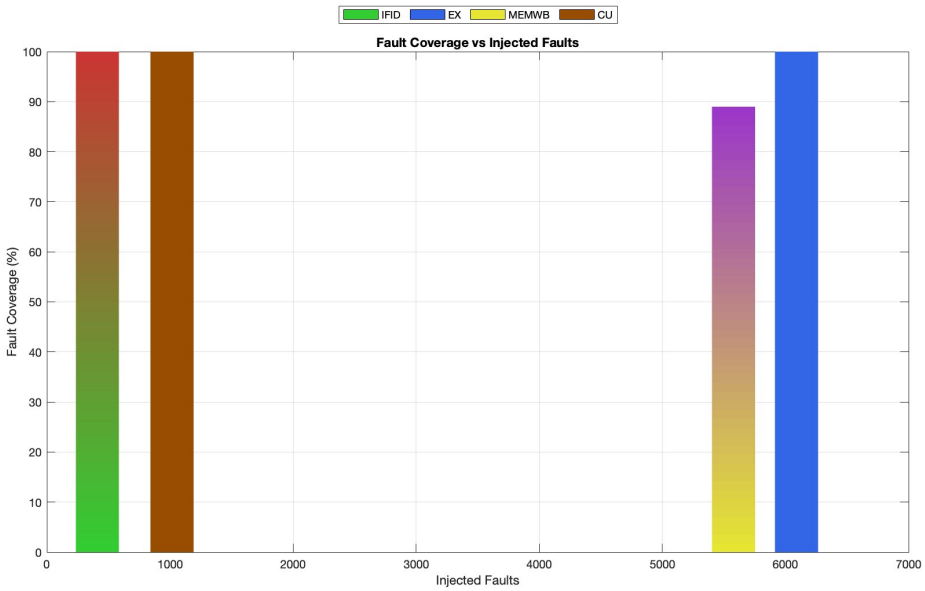


Figure 15: Caption

5 Conclusion

For this step we worked on different components of the DLX. This time we aimed to the maximize coverage, by analyzing coverage reports and coming up with solutions to increase it as much as possible. Both RTL and post-synthesis coverages were examined, and as a general rule of thumb, synthesizing the design typically introduces some pain points, and is unfeasible to analyze post-synthesis netlist with the aim of improving coverage by modifying it. It was however possible in a few cases to take a guess about what the synthesizer was doing behind the scences that was driving the obtained results, allowing a critical analysis of them and further enhancement.

Regarding the testing assignment, fault simulating with the first Bash-based approach resulted in longer simulation times, and also hancrafted fault lists, which results in lower overall excited faults in a campaign. On the other hand, the second, Class-based faster approach, with the custom script to generate the fault list file, lead to very fast fault simulation times, allowing us to simulate every fault which was generated.

Acronyms

UVM Universal Verification Methodology

SV SystemVerilog